

Praktikum 5: Sinkronisasi Sistem Operasi

Informasi Praktikum

- **Mata Kuliah:** Sistem Operasi
 - **Sistem Operasi:** Linux Ubuntu
 - **Bahasa Pemrograman:** C
 - **Dosen Pengampu:** Triawan Adi Cahyanto, M.Kom
 - **Asisten Dosen:** Atidhira Habibillah dan Taqiyyuddin
-

Pengantar

Praktikum ini akan membantu Anda memahami konsep sinkronisasi dalam sistem operasi melalui praktik langsung. Anda akan belajar bagaimana mengatasi masalah ketika beberapa proses/thread mencoba mengakses data yang sama secara bersamaan.

Analogi Sederhana: Bayangkan Anda dan teman Anda mencoba mengedit dokumen yang sama di waktu bersamaan. Tanpa aturan, tulisan kalian bisa saling menimpa dan berantakan. Sinkronisasi adalah "aturan main" agar tidak terjadi kekacauan.

Tujuan Praktikum

Setelah praktikum ini, Anda diharapkan dapat:

1. Memahami masalah race condition (kondisi balapan)
 2. Menggunakan Mutex untuk melindungi data bersama
 3. Menggunakan Semaphore untuk mengatur akses ke sumber daya
 4. Menyelesaikan masalah Producer-Consumer
 5. Memahami cara menghindari deadlock
-

Persiapan

Software yang Dibutuhkan:

```
# Update sistem
sudo apt update

# Install compiler C dan library pthread
sudo apt install build-essential

# Verifikasi instalasi
gcc --version
```

Struktur Folder:

```
# Buat folder untuk praktikum
mkdir ~/praktikum-sinkronisasi
cd ~/praktikum-sinkronisasi

# Buat subfolder untuk setiap percobaan
mkdir percobaan1 percobaan2 percobaan3 percobaan4
```

Topik 1: Race Condition (Masalah Tanpa Sinkronisasi)

Tujuan:

Melihat langsung apa yang terjadi ketika tidak ada sinkronisasi

Konsep:

Race condition terjadi ketika dua atau lebih thread mengubah data yang sama secara bersamaan, menghasilkan hasil yang tidak dapat diprediksi.

Analogi: Dua orang menarik uang dari ATM yang sama di waktu bersamaan, bisa-bisa saldo terdebit dua kali!

Kode Program:

```
// File: percobaan1/race_condition.c

#include <stdio.h>
#include <pthread.h>

// Variabel bersama (shared variable)
int counter = 0;

// Fungsi yang akan dijalankan oleh thread
void* increment_counter(void* arg) {
    for(int i = 0; i < 100000; i++) {
        counter++; // Operasi tidak aman!
    }
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    printf("Counter awal: %d\n", counter);

    // Membuat 2 thread yang menjalankan fungsi yang sama
    pthread_create(&thread1, NULL, increment_counter, NULL);
    pthread_create(&thread2, NULL, increment_counter, NULL);

    // Menunggu kedua thread selesai
```

```
pthread_join(thread1, NULL);  
pthread_join(thread2, NULL);  
  
printf("Counter akhir: %d\n", counter);  
printf("Seharusnya: 200000\n");  
  
return 0;  
}
```

Cara Menjalankan:

```
cd ~/praktikum-sinkronisasi/percobaan1  
  
# Compile program  
gcc race_condition.c -o race_condition -pthread  
  
# Jalankan beberapa kali untuk melihat hasil berbeda  
./race_condition  
./race_condition  
./race_condition
```

Hasil yang Diharapkan:

Anda akan melihat nilai counter akhir **BUKAN 200000** dan **berbeda-beda** setiap kali dijalankan. Ini adalah race condition!

Jawab pertanyaan berikut untuk ditulis di laporan praktikum

1. Mengapa hasilnya tidak 200000?
2. Mengapa hasilnya berbeda-beda setiap kali dijalankan?
3. Apa yang terjadi ketika dua thread mengakses counter bersamaan?

Topik 2: Mutex Lock (Solusi Race Condition)

Tujuan:

Menggunakan Mutex untuk melindungi critical section

Konsep:

Mutex (Mutual Exclusion) adalah kunci digital. Hanya thread yang memegang kunci yang bisa mengakses data. Thread lain harus menunggu.

Analogi: Toilet umum dengan satu kunci. Hanya orang yang memegang kunci yang bisa masuk. Yang lain harus antri di luar.

Kode Program:

```
// File: percobaan2/mutex_solution.c

#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock; // Deklarasi mutex

void* increment_counter(void* arg) {
    for(int i = 0; i < 100000; i++) {
        // Kunci pintu sebelum masuk
        pthread_mutex_lock(&lock);

        // CRITICAL SECTION (bagian yang dilindungi)
        counter++;

        // Buka kunci setelah selesai
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    // Inisialisasi mutex
    pthread_mutex_init(&lock, NULL);

    printf("Counter awal: %d\n", counter);

    pthread_create(&thread1, NULL, increment_counter, NULL);
    pthread_create(&thread2, NULL, increment_counter, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Counter akhir: %d\n", counter);
    printf("Seharusnya: 200000\n");

    // Hancurkan mutex setelah selesai
    pthread_mutex_destroy(&lock);

    return 0;
}
```

Cara Menjalankan:

```
cd ~/praktikum-sinkronisasi/percobaan2

gcc mutex_solution.c -o mutex_solution -pthread
```

```
# Jalankan beberapa kali
./mutex_solution
./mutex_solution
./mutex_solution
```

Hasil yang Diharapkan:

Counter akhir **SELALU 200000** setiap kali dijalankan!

Jawab pertanyaan berikut

1. Mengapa sekarang hasilnya selalu benar?
2. Apa yang terjadi ketika thread1 mengunci mutex dan thread2 mencoba mengaksesnya?
3. Bagaimana jika kita lupa memanggil `pthread_mutex_unlock()`?

Topik 3: Semaphore (Mengatur Akses Terbatas)

Tujuan:

Menggunakan Semaphore untuk membatasi jumlah akses bersamaan

Konsep:

Semaphore seperti penjaga pintu yang mengizinkan sejumlah orang tertentu masuk. Misalnya, restoran dengan 3 meja hanya bisa melayani 3 kelompok sekaligus.

Analogi: Parkiran dengan 5 slot. Mobil keenam harus menunggu sampai ada yang keluar.

Kode Program:

```
// File: percobaan3/semaphore_parking.c

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define PARKING_SLOTS 3 // Jumlah tempat parkir
#define TOTAL_CARS 10  // Jumlah mobil yang ingin parkir

sem_t parking; // Semaphore untuk parkir

void* car(void* arg) {
    int car_id = *(int*)arg;

    printf("Mobil %d mencoba masuk parkir...\n", car_id);

    // Coba dapatkan slot parkir
    sem_wait(&parking);
```

```
printf("Mobil %d BERHASIL parkir!\n", car_id);

// Simulasi waktu parkir (2 detik)
sleep(2);

printf("Mobil %d keluar dari parkiran\n", car_id);

// Lepaskan slot parkir
sem_post(&parking);

return NULL;
}

int main() {
pthread_t threads[TOTAL_CARS];
int car_ids[TOTAL_CARS];

// Inisialisasi semaphore dengan 3 slot
sem_init(&parking, 0, PARKING_SLOTS);

printf("Parkiran dibuka dengan %d slot!\n\n", PARKING_SLOTS);

// Buat thread untuk setiap mobil
for(int i = 0; i < TOTAL_CARS; i++) {
    car_ids[i] = i + 1;
    pthread_create(&threads[i], NULL, car, &car_ids[i]);
    usleep(100000); // Jeda 0.1 detik antar mobil
}

// Tunggu semua mobil selesai
for(int i = 0; i < TOTAL_CARS; i++) {
    pthread_join(threads[i], NULL);
}

printf("\n Semua mobil selesai!\n");

// Hancurkan semaphore
sem_destroy(&parking);

return 0;
}
```

Cara Menjalankan:

```
cd ~/praktikum-sinkronisasi/percobaan3

gcc semaphore_parking.c -o semaphore_parking -pthread

./semaphore_parking
```

Hasil yang Diharapkan:

- Maksimal 3 mobil parkir bersamaan
- Mobil ke-4 dan seterusnya harus menunggu
- Ketika ada yang keluar, mobil berikutnya bisa masuk

Jawab pertanyaan berikut

1. Apa yang terjadi jika kita ubah PARKING_SLOTS menjadi 1?
2. Bagaimana jika kita ubah menjadi 10?
3. Apa perbedaan Semaphore dengan Mutex?

Topik 4: Producer-Consumer Problem

Tujuan:

Menyelesaikan masalah klasik sinkronisasi

Konsep:

Producer membuat data, Consumer mengkonsumsi data. Buffer (tempat penyimpanan sementara) memiliki kapasitas terbatas.

Analogi: Dapur restoran (Producer) membuat makanan, Pelayan (Consumer) mengambil makanan. Meja penyajian hanya muat 5 piring.

Kode Program:

```
// File: percobaan4/producer_consumer.c

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0; // Index untuk producer
int out = 0; // Index untuk consumer

sem_t empty; // Semaphore untuk slot kosong
sem_t full; // Semaphore untuk slot terisi
pthread_mutex_t mutex; // Mutex untuk buffer

void* producer(void* arg) {
    int item;
    for(int i = 0; i < 10; i++) {
        item = i + 1; // Buat item (data)
```

```
// Tunggu sampai ada slot kosong
sem_wait(&empty);

// Kunci buffer
pthread_mutex_lock(&mutex);

// CRITICAL SECTION
buffer[in] = item;
printf("Producer membuat item %d (slot %d)\n", item, in);
in = (in + 1) % BUFFER_SIZE;

// Buka kunci buffer
pthread_mutex_unlock(&mutex);

// Beritahu ada item baru
sem_post(&full);

sleep(1); // Simulasi waktu produksi
}
return NULL;
}

void* consumer(void* arg) {
    int item;
    for(int i = 0; i < 10; i++) {
        // Tunggu sampai ada item
        sem_wait(&full);

        // Kunci buffer
        pthread_mutex_lock(&mutex);

        // CRITICAL SECTION
        item = buffer[out];
        printf("Consumer mengambil item %d (slot %d)\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        // Buka kunci buffer
        pthread_mutex_unlock(&mutex);

        // Beritahu ada slot kosong
        sem_post(&empty);

        sleep(2); // Simulasi waktu konsumsi (lebih lambat)
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;

    // Inisialisasi semaphore
    sem_init(&empty, 0, BUFFER_SIZE); // Awalnya semua slot kosong
    sem_init(&full, 0, 0);           // Awalnya tidak ada item
    pthread_mutex_init(&mutex, NULL);
```



```
printf("Simulasi Producer-Consumer dimulai!\n");
printf("Buffer size: %d\n\n", BUFFER_SIZE);

// Buat thread producer dan consumer
pthread_create(&prod, NULL, producer, NULL);
pthread_create(&cons, NULL, consumer, NULL);

// Tunggu kedua thread selesai
pthread_join(prod, NULL);
pthread_join(cons, NULL);

printf("\n Simulasi selesai!\n");

// Bersihkan
sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

return 0;
}
```

Cara Menjalankan:

```
cd ~/praktikum-sinkronisasi/percobaan4

gcc producer_consumer.c -o producer_consumer -pthread

./producer_consumer
```

Hasil yang Diharapkan:

- Producer membuat item secara berkala
- Consumer mengambil item (lebih lambat)
- Buffer tidak overflow atau underflow
- Semua item diproduksi dan dikonsumsi dengan benar

Jawab pertanyaan berikut

1. Apa yang terjadi jika buffer penuh dan producer mencoba menambah item?
2. Apa yang terjadi jika buffer kosong dan consumer mencoba mengambil item?
3. Mengapa kita butuh mutex dan semaphore?

Topik 5: Deadlock (Masalah Kebuntuan)

Tujuan:

Memahami dan menghindari deadlock

Konsep:

Deadlock terjadi ketika dua proses saling menunggu sumber daya yang dipegang oleh proses lain.

Analogi: Dua mobil di jalan sempit saling menunggu yang lain mundur dulu.

Kode Program (Contoh Deadlock):

```
// File: percobaan4/deadlock_example.c

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1, lock2;

void* thread1_func(void* arg) {
    printf("Thread 1: Mencoba kunci lock1...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 1: Mendapat lock1! \n");

    sleep(1); // Simulasi pekerjaan

    printf("Thread 1: Mencoba kunci lock2...\n");
    pthread_mutex_lock(&lock2);
    printf("Thread 1: Mendapat lock2! \n");

    // Critical section
    printf("Thread 1: Selesai!\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

void* thread2_func(void* arg) {
    printf("Thread 2: Mencoba kunci lock2...\n");
    pthread_mutex_lock(&lock2);
    printf("Thread 2: Mendapat lock2! \n");

    sleep(1); // Simulasi pekerjaan

    printf("Thread 2: Mencoba kunci lock1...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 2: Mendapat lock1! \n");

    // Critical section
    printf("Thread 2: Selesai!\n");

    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);
}
```

```

    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock1, NULL);
    pthread_mutex_init(&lock2, NULL);

    printf("Demonstrasi DEADLOCK!\n\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    // Program akan HANG di sini!
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock1);
    pthread_mutex_destroy(&lock2);

    return 0;
}

```

Solusi Deadlock:

```

// File: percobaan4/deadlock_solution.c

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1, lock2;

void* thread1_func(void* arg) {
    // SOLUSI: Kedua thread mengunci dalam urutan yang SAMA
    pthread_mutex_lock(&lock1);
    printf("Thread 1: Mendapat lock1! \n");

    sleep(1);

    pthread_mutex_lock(&lock2);
    printf("Thread 1: Mendapat lock2! \n");

    printf("Thread 1: Selesai!\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

```

```

}

void* thread2_func(void* arg) {
    // KUNCI PENTING: Urutan yang sama dengan thread1
    pthread_mutex_lock(&lock1); // Bukan lock2!
    printf("Thread 2: Mendapat lock1! \n");

    sleep(1);

    pthread_mutex_lock(&lock2);
    printf("Thread 2: Mendapat lock2! \n");

    printf("Thread 2: Selesai!\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock1, NULL);
    pthread_mutex_init(&lock2, NULL);

    printf("Solusi Deadlock: Resource Ordering\n\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\n Selesai tanpa deadlock!\n");

    pthread_mutex_destroy(&lock1);
    pthread_mutex_destroy(&lock2);

    return 0;
}

```

Cara Menjalankan:

```

# Deadlock (akan hang)
gcc deadlock_example.c -o deadlock_example -pthread
./deadlock_example
# Tekan Ctrl+C untuk menghentikan

# Solusi

```

```
gcc deadlock_solution.c -o deadlock_solution -pthread
./deadlock_solution
```

Laporan Praktikum

Buat laporan yang berisi:

1. Judul dan Identitas

- Nama, NIM, Kelas
- Judul: Praktikum Sinkronisasi Sistem Operasi

2. Tujuan Praktikum

Tuliskan dengan kata-kata sendiri tujuan dari praktikum ini

3. Hasil Percobaan

Untuk setiap percobaan, sertakan:

- Screenshot output program
- Penjelasan apa yang terjadi
- Jawaban dari pertanyaan analisis

4. Eksperimen Tambahan

Coba modifikasi salah satu program:

- Ubah jumlah thread
- Ubah ukuran buffer
- Tambahkan lebih banyak producer/consumer
- Screenshot dan jelaskan hasilnya

5. Kesimpulan

- Apa yang Anda pelajari?
- Kapan harus menggunakan Mutex vs Semaphore?
- Mengapa sinkronisasi penting?

Tugas Kuliah

SISTEM BOOKING HOTEL (Akhiran NIM Ganjil)

Deskripsi Masalah:

Anda diminta membuat simulasi sistem booking hotel dengan kondisi:

- Hotel memiliki **5 kamar** yang bisa di-booking
- Ada **10 customer** yang mencoba booking secara bersamaan (10 threads)

- Setiap customer akan:
 1. Mencoba booking kamar (butuh waktu random 1-3 detik)
 2. Jika berhasil, "menginap" selama 5 detik
 3. Check-out dan melepaskan kamar
 4. Jika gagal (kamar penuh), customer menunggu 2 detik lalu coba lagi
- Customer **maksimal mencoba 3 kali**, jika gagal terus maka menyerah

Requirements:

1. Gunakan **Semaphore** untuk mengatur jumlah kamar tersedia
2. Gunakan **Mutex** untuk melindungi operasi print (agar output tidak berantakan)
3. Tampilkan informasi:
 - Customer ID berapa yang berhasil booking
 - Kamar mana yang ditempati
 - Berapa kamar yang masih tersedia
 - Customer yang gagal booking
4. **Output harus rapi dan mudah dibaca** (gunakan emoji atau format yang jelas)

Contoh Output yang Diharapkan:

```
=== SISTEM BOOKING HOTEL ===
Total Kamar: 5

[10:30:15] 🏃 Customer-1 mencoba booking...
[10:30:15] 🏃 Customer-2 mencoba booking...
[10:30:15] ✅ Customer-1 berhasil booking! (4 kamar tersisa)
[10:30:16] ✅ Customer-2 berhasil booking! (3 kamar tersisa)
[10:30:16] 🏃 Customer-3 mencoba booking...
[10:30:16] ❌ Customer-10 gagal booking (kamar penuh), mencoba lagi...
[10:30:20] 📄 Customer-1 check-out (4 kamar tersisa)
...
```

Yang Harus Dikumpulkan:

1. Source code lengkap dengan komentar
2. Screenshot output program
3. Penjelasan bagaimana Anda mengatasi race condition
4. Analisis: Apa yang terjadi jika tidak menggunakan sinkronisasi?

DINING PHILOSOPHERS PROBLEM (Akhiran NIM Genap)

Deskripsi Masalah:

Implementasikan solusi untuk masalah klasik 5 filsuf makan dengan **mencegah deadlock!**

Kondisi:

- 5 filsuf duduk melingkar

- 5 sumpit (1 antara tiap filsuf)
- Filsuf perlu 2 sumpit untuk makan
- Filsuf berpikir → lapar → makan → berpikir (siklus)

Requirements:

Deadlock Demonstration:

1. Buat program yang **MENIMBULKAN DEADLOCK**
2. Semua filsuf mengambil sumpit kiri dulu, lalu kanan
3. Tampilkan saat program hang/deadlock
4. **Jelaskan mengapa deadlock terjadi**

Deadlock Prevention:

1. Implementasikan **minimal 2 solusi berbeda** untuk mencegah deadlock:
 - Solusi A: Resource Ordering (filsuf genap ambil kiri dulu, ganjil ambil kanan dulu)
 - Solusi B: Limiting Philosophers (max 4 filsuf makan bersamaan)
 - Solusi C: Asymmetric Solution (1 filsuf urutan berbeda)
2. Jalankan masing-masing solusi selama 60 detik
3. Bandingkan **efisiensi** (berapa kali masing-masing filsuf berhasil makan)

Contoh Output:

```

=== DINING PHILOSOPHERS PROBLEM ===
Solusi: Resource Ordering

[00:01] 🧠 Filsuf-1 sedang berpikir...
[00:03] 🕒 Filsuf-1 lapar, mencoba ambil sumpit...
[00:04] // Filsuf-1 mengambil sumpit kiri (0)
[00:04] // Filsuf-1 mengambil sumpit kanan (1)
[00:04] 🍴 Filsuf-1 sedang MAKAN...
[00:09] // Filsuf-1 menaruh kedua sumpit
[00:09] 🧠 Filsuf-1 sedang berpikir...
...

=== STATISTIK (60 detik) ===
Filsuf-1: Makan 8 kali
Filsuf-2: Makan 7 kali
Filsuf-3: Makan 8 kali
Filsuf-4: Makan 7 kali
Filsuf-5: Makan 8 kali
☒ Tidak ada deadlock!
☒ Distribusi fairness: BAIK

```

Yang Harus Dikumpulkan:

1. 3 program terpisah (deadlock version + 2 solusi)
2. Screenshot masing-masing output

3. Tabel perbandingan efisiensi dan fairness
 4. Analisis mendalam:
 - Mengapa deadlock bisa terjadi?
 - Bagaimana masing-masing solusi bekerja?
 - Solusi mana yang paling adil (fair)?
 - Apakah ada kemungkinan starvation?
-

TEMPLATE TUGAS

Laporan harus berisi (minimal 5 halaman):

1. Cover

- Judul, Nama, NIM, Kelas, Tanggal

2. Daftar Isi

3. Soal Pemrograman

Untuk setiap soal:

3.1. Analisis Masalah

- Pemahaman Anda tentang soal
- Identifikasi masalah sinkronisasi
- Desain solusi

3.2. Implementasi

- Penjelasan algoritma
- Flowchart/diagram
- Potongan kode penting dengan penjelasan
- Mengapa memilih mekanisme tertentu

3.3. Testing & Output

- Screenshot output dengan berbagai skenario
- Test case yang dicoba
- Hasil yang didapat

3.4. Analisis Hasil

- Apakah program berjalan sesuai ekspektasi?
- Masalah yang ditemui dan solusinya
- Performa program
- Pelajaran yang didapat

4. Kesimpulan

- Rangkuman apa yang dipelajari
- Tantangan yang dihadapi
- Saran pengembangan

5. Referensi

- Buku/artikel yang dirujuk
- Website yang digunakan

TIPS SUKSES

Untuk Coding:

1. **Mulai dari yang sederhana** - Test dengan 2 thread dulu sebelum 10 thread
2. **Compile incremental** - Jangan langsung buat semua, test per bagian
3. **Debug dengan print statement** - Tampilkan state program
4. **Backup kode** - Gunakan Git atau simpan versi berbeda
5. **Test corner cases** - Apa yang terjadi jika thread 0, 1, atau sangat banyak?

Untuk Isi Penyusunan Tugas:

1. **Screenshot informatif** - Beri penjelasan/arrow pada bagian penting
2. **Gunakan diagram** - Flowchart, state diagram, sequence diagram
3. **Analisis kritis** - Jangan hanya deskripsi, berikan analisis
4. **Format konsisten** - Font, spacing, heading yang rapi
5. **Sitasi yang benar** - Jangan plagiat, cantumkan sumber

Tips & Troubleshooting

Compile Error: "undefined reference to pthread_create"

Solusi: Tambahkan flag `-pthread` saat compile

```
gcc program.c -o program -pthread
```

Program Hang (Tidak Selesai)

Kemungkinan: Deadlock atau lupa unlock mutex **Solusi:** Periksa apakah setiap `lock()` ada pasangan `unlock()`-nya

Hasil Masih Race Condition

Kemungkinan: Lupa mengunci critical section **Solusi:** Pastikan semua akses ke variabel bersama dilindungi mutex

Semaphore Error

Solusi: Jangan lupa `#include <semaphore.h>`

Referensi

1. POSIX Threads Programming: <https://computing.llnl.gov/tutorials/pthreads/>
2. Man pages: `man pthread_mutex_lock`, `man sem_wait`
3. Buku: "Operating System Concepts" - Silberschatz